Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

2. Learning Outcomes

Students will be able to

1. Explain the convolution operation.
2. Compute output dimensions using stride and padding.
3. Visualize feature maps.
4. Implement max pooling and average pooling.
5. Calculate CNN parameters.
6. Build a CNN for image classification.
7. Evaluate CNN performance.

3. Background Theory

A Convolutional Neural Network consists of

- Convolution Layer
- Activation Layer
- Pooling Layer
- Fully Connected Layer

The convolution operation is

$$Y(i,j) = \sum_m \sum_n X(i+m, j+n) K(m,n)$$

where

- X : Input Image
- K : Kernel (Filter)
- Y : Feature Map

The output feature map size is

$$\frac{N - F + 2P}{S} + 1$$

where

N Input Size

F Filter Size

P Padding

S Stride

3.1 Common Convolution Kernels

A **kernel** (or **filter**) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions. During convolution, the kernel is multiplied element-wise with the corresponding image pixels, and the results are summed to produce a single value in the output feature map. Different kernels highlight different image characteristics.

1. Identity Kernel

The identity kernel preserves the original image without modifying its pixel values.

$$\begin{matrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{matrix}$$

Purpose:

- Preserves the original image.
- Used for understanding the convolution operation.

2. Sobel-X Kernel (Vertical Edge Detection)

The Sobel-X kernel detects **vertical edges** by measuring intensity changes along the horizontal direction.

$$\begin{matrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{matrix}$$

Applications:

- Object detection
- Lane detection
- Face recognition

3. Sobel-Y Kernel (Horizontal Edge Detection)

The Sobel-Y kernel detects **horizontal edges** by measuring intensity changes along the vertical direction.

$$\begin{matrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{matrix}$$

Applications:

- Text detection
- Boundary extraction
- Medical image analysis

4. Laplacian Kernel

The Laplacian kernel detects edges in all directions using the second-order derivative of image intensity.

$$\begin{matrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{matrix}$$

Applications:

- Edge enhancement
- Satellite image analysis
- Medical imaging

5. Sharpening Kernel

This kernel enhances fine details by emphasizing high-frequency components.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Image enhancement
- Optical Character Recognition (OCR)
- Face recognition

6. Box Blur Kernel

The Box Blur kernel smooths an image by replacing each pixel with the average of its neighboring pixels.

$$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Applications:

- Noise removal
- Image smoothing
- Image preprocessing

7. Gaussian Blur Kernel

Gaussian blur smooths the image while preserving the overall structure better than a simple average filter.

$$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Noise reduction
- Computer vision preprocessing
- Feature extraction

8. Emboss Kernel

The Emboss kernel creates a three-dimensional appearance by emphasizing directional changes in intensity.

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

Applications:
- Texture analysis
- Artistic image effects

9. Outline Detection Kernel

This kernel highlights object boundaries by emphasizing regions with rapid intensity changes.

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Applications:
- Image segmentation
- Boundary detection
- Shape extraction

10. Motion Blur Kernel

This kernel simulates motion blur along the diagonal direction.

$$\frac{1}{5}\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Applications:
- Motion simulation
- Image restoration
- Video processing

Summary of Common Kernels

Kernel	Size	Purpose	Applications
Identity	3 × 3	Preserve original image	Baseline comparison
Sobel-X	3 × 3	Detect vertical edges	Object detection
Sobel-Y	3 × 3	Detect horizontal edges	Boundary detection
Laplacian	3 × 3	Detect edges in all directions	Medical imaging
Sharpen	3 × 3	Enhance details	Image enhancement
Box Blur	3 × 3	Smooth image	Noise removal
Gaussian Blur	3 × 3	Reduce noise	Computer vision
Emboss	3 × 3	3D effect	Artistic filtering
Outline	3 × 3	Boundary extraction	Segmentation
Motion Blur	5 × 5	Simulate motion	Video processing

4. Numerical Example 1: Convolution

Consider the following input image.

$$X = \begin{matrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{matrix}$$

Kernel

$$K = \begin{matrix} 1 & 0 \\ 0 & 1 \end{matrix}$$

Output
First position

$$1(1) + 2(0) + 4(0) + 5(1) = 6$$

Second position

$$2(1) + 3(0) + 5(0) + 6(1) = 8$$

Third position

$$4(1) + 5(0) + 7(0) + 8(1) = 12$$

Fourth position

$$5(1) + 6(0) + 8(0) + 9(1) = 14$$

Feature Map

$$\begin{matrix} 6 & 8 \\ 12 & 14 \end{matrix}$$

5. Numerical Example 2: Max Pooling

Feature map

$$\begin{matrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{matrix}$$

Using a 2 × 2 max pooling filter,

Window 1

$$\begin{bmatrix} 1 & 5 \\ 7 & 8 \end{bmatrix} \rightarrow 8$$

Window 2

$$\begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix} \rightarrow 3$$

Window 3

$$\begin{bmatrix} 4 & 6 \\ 2 & 3 \end{bmatrix} \rightarrow 6$$

Window 4

$$\begin{bmatrix} 9 & 5 \\ 1 & 8 \end{bmatrix} \rightarrow 9$$

Output

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

6. Numerical Example 3: Parameter Calculation

Suppose

Input Image

$$32 \times 32 \times 3$$

Convolution Layer

- Filters = 16
- Kernel = 3 × 3

Parameters

$$(3 \times 3 \times 3 + 1) \times 16$$

$$= (27 + 1) \times 16$$

= 448

Therefore,
Total trainable parameters = 448.

7. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Classes : 10
- Image Size : $32 \times 32 \times 3$

8. Experimental Procedure

Task 1

Load CIFAR-10.

- Display ten sample images.
- Print dataset dimensions.
- Plot class distribution.

Task 2

Implement a convolution layer.
Compare

- Kernel = 3×3
- Kernel = 5×5
- Kernel = 7×7

Record feature map sizes.

Task 3

Study hyperparameters.
Compare

- Stride = 1
- Stride = 2
- Padding = Same
- Padding = Valid

Compute output dimensions.

Task 4

Visualize feature maps after the first convolution layer.
Display at least eight feature maps.

Task 5

Compare

- Max Pooling
- Average Pooling

Compare output size and accuracy.

Task 6

Construct the following CNN.

Input → Conv → ReLU → MaxPool → Conv → ReLU → MaxPool → Flatten → Dense → Softmax

Train for

- Optimizer : Adam
- Epochs : 20
- Batch Size : 32

Task 7

Evaluate

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

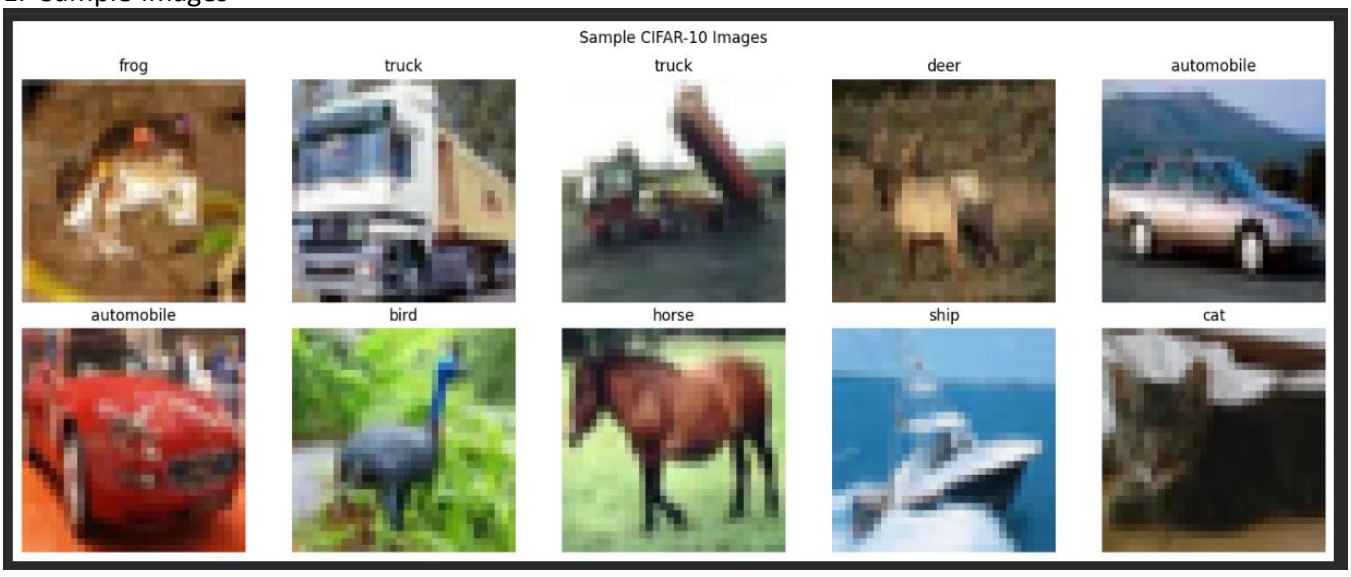
9. source code

https://github.com/SUJJU003/Deep-Learning-Lab/blob/main/Experiment_3_CNN.ipynb

10. Mandatory Plots

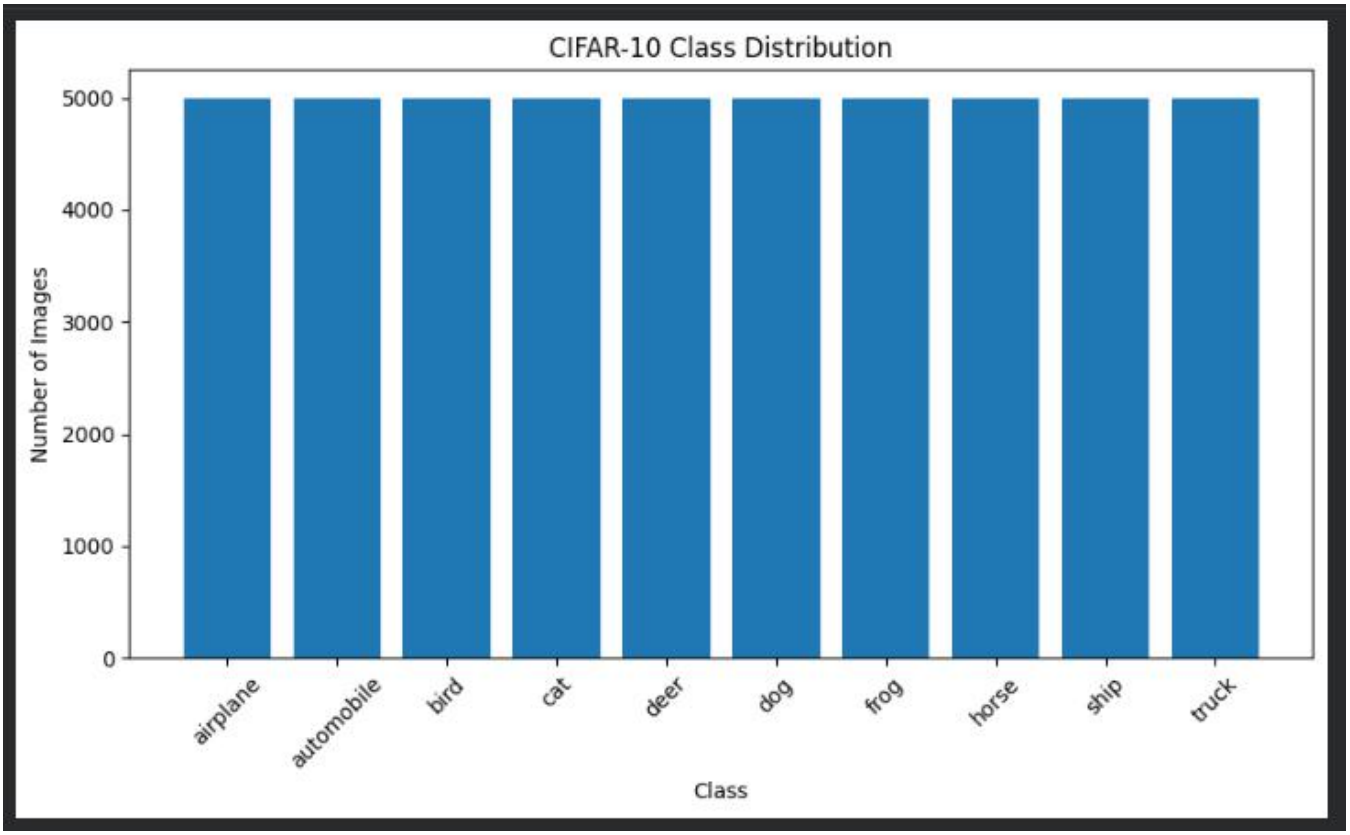
Generate

1. Sample Images



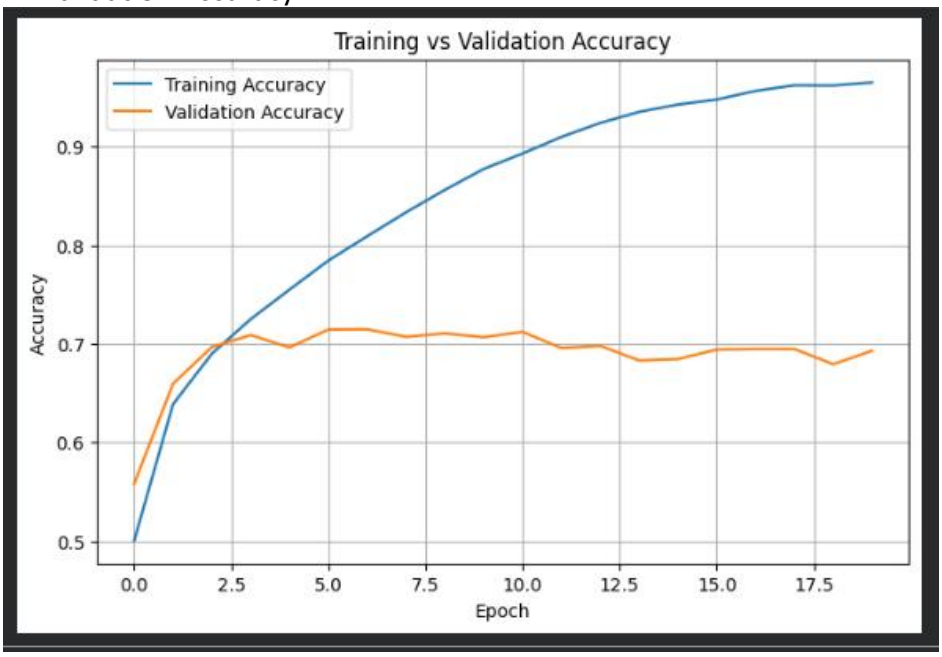
The sample images represent different categories from the CIFAR-10 dataset, including animals, vehicles, and other objects. Each image has a resolution of 32×32 pixels with three RGB channels.

2. Class Distribution



The class distribution is approximately balanced, with each of the ten CIFAR-10 classes containing a similar number of images. Therefore, the model is not significantly affected by class imbalance during training.

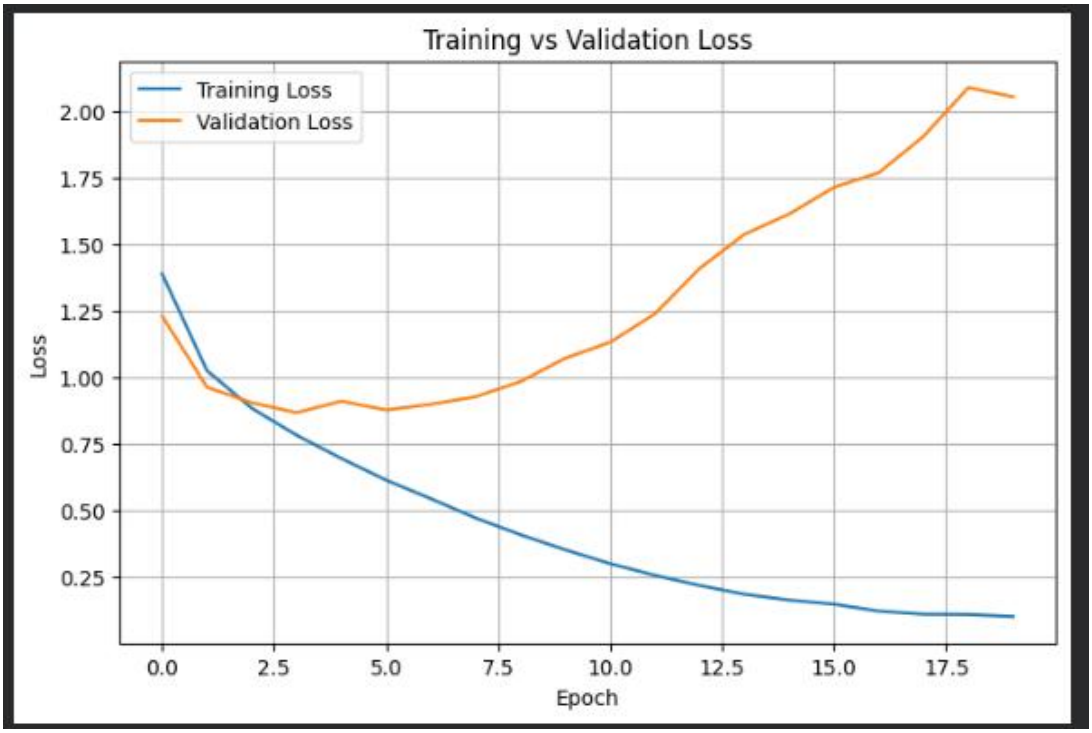
3. Training Accuracy
4. Validation Accuracy



The training accuracy generally increases as the number of epochs increases, indicating that the CNN is learning useful patterns from the training images. The improvement in accuracy demonstrates the effectiveness of convolutional layers in extracting image features.

The validation accuracy indicates how well the trained CNN generalizes to unseen images. A relatively stable validation accuracy compared with training accuracy suggests that the model is learning features that generalize beyond the training data.

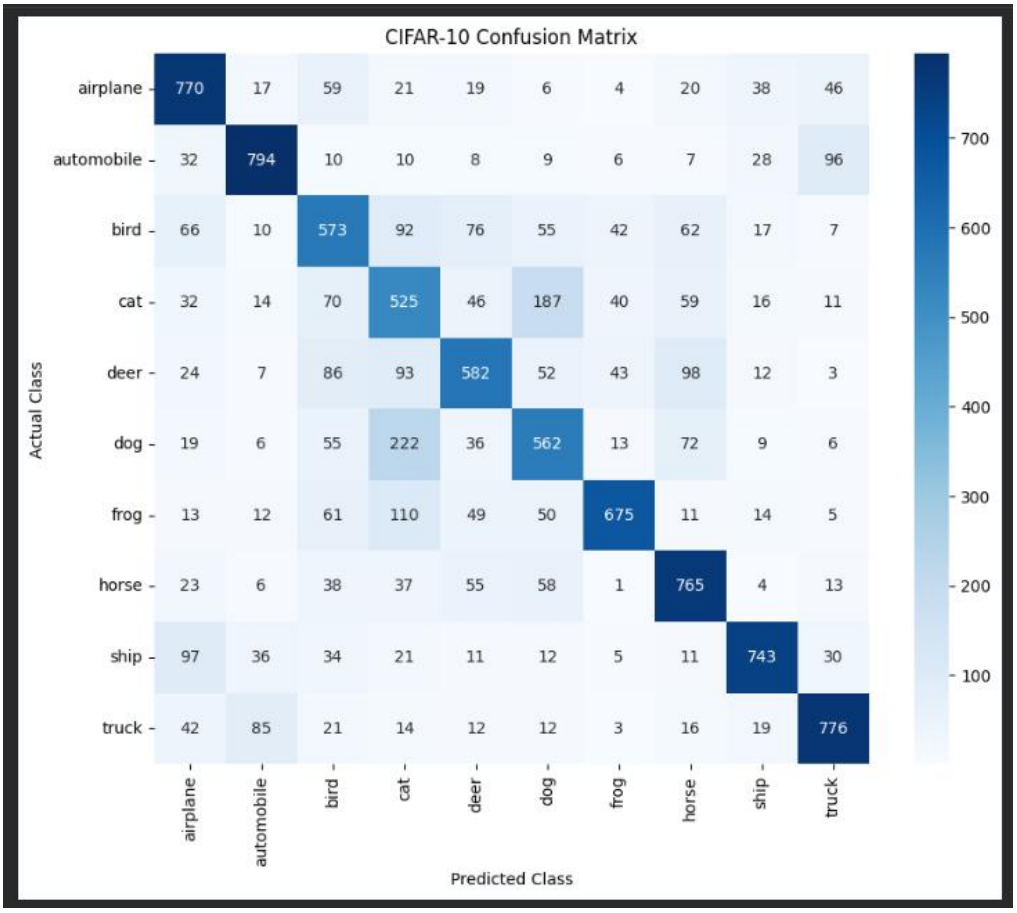
5. Training Loss
6. Validation Loss



The training loss generally decreases with increasing epochs, showing that the optimization process is successfully reducing the classification error on the training data. This indicates that the CNN is progressively learning better feature representations.

The validation loss represents the classification error on unseen validation images. A decreasing or relatively stable validation loss indicates improving generalization, whereas an increasing validation loss along with decreasing training loss would indicate overfitting.

7. Feature Maps
8. Confusion Matrix

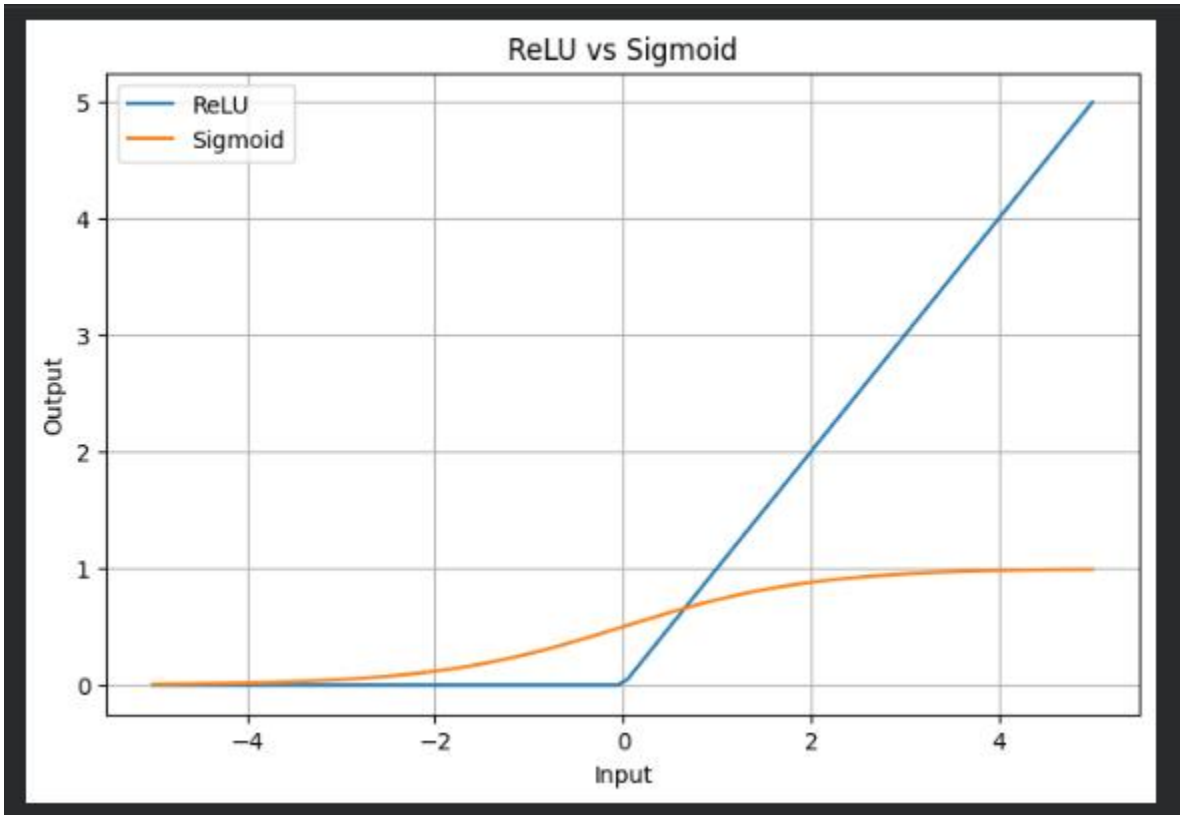


The feature maps show the spatial responses produced by the filters in the first convolutional layer. Different filters generate different activation patterns, demonstrating how convolutional layers transform input images into feature representations.

The confusion matrix shows the number of correctly and incorrectly classified images for each CIFAR-10 class. The model performs relatively well on classes such as automobile, ship, and truck, while greater confusion is observed among visually similar animal classes.

11. Additional Exercises

1. Calculate the output size for a 64 × 64 image using a 5 × 5 kernel with stride 2 and padding 2.
2. Compute the trainable parameters of a convolution layer having 64 filters of size 3 × 3 and an RGB input.
3. Compare ReLU and Sigmoid activation functions.
4. Replace Max Pooling with Average Pooling and compare the results.
5. Increase the number of convolution filters from 16 to 64 and analyze the effect on accuracy and computation time.



12. Results

Record

- Training Accuracy
- Testing Accuracy
- Precision
- Recall
- F1-score
- Number of Parameters

```
==================================================
          CNN EXPERIMENT RESULTS
==================================================
Training Accuracy : 0.9653
Testing Accuracy  : 0.6765
Precision         : 0.6833
Recall            : 0.6765
F1 Score          : 0.6782
Total Parameters  : 545098
==================================================
```

13. Discussion

Answer the following.

1. Why is convolution preferred over fully connected layers for images?

Convolution is preferred because it preserves the spatial structure of images and extracts local features using small kernels. It also uses the same filter across different regions of the image, reducing the number of parameters.
Example: A 3×3 kernel can detect an edge wherever that edge appears in an image, rather than learning separate weights for every location.

2. How does stride affect the feature map size?

Stride determines how many pixels the kernel moves at each step. A larger stride produces a smaller feature map because the kernel is applied at fewer positions.

For example, in your experiment with a 3×3 kernel:

Stride 1 with valid padding → 30×30
Stride 2 with valid padding → 15×15

3. What is the role of padding?

Padding adds extra pixels around the input image before convolution. It helps control the output dimensions and allows information near the image boundaries to be processed.
Example: With a 3×3 kernel and stride 1, same padding keeps a 32×32 input as 32×32, while valid padding reduces it to 30×30

4. Why is pooling used?

Pooling reduces the spatial dimensions of feature maps, which decreases computational requirements and retains important information. It also helps make the representation less sensitive to small spatial changes.
Example: Your 2×2 pooling with stride 2 changed a 16×16 feature map into an 8×8 feature map.

5. How do feature maps represent image characteristics?

A feature map represents the response of a particular convolutional filter to different regions of an image. Different filters respond to different visual patterns, allowing the CNN to build representations of image characteristics.

Example: One filter may respond strongly to an edge, while another may respond to a texture or pattern.

6. Why do CNNs require fewer parameters than MLPs?

CNNs require fewer parameters because they use **local connectivity** and **weight sharing**. A convolutional filter uses the same set of weights across the entire image, whereas an MLP typically uses separate weights for every connection between input pixels and neurons.

Example: A 3×3 filter on an RGB image with 32 filters needs only $(3\times3\times3+1)\times32=896$ parameters, whereas connecting all $32\times32\times3=3072$ input pixels directly to a dense layer would require far more parameters.

14. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. TensorFlow Documentation.
   CIFAR-10 Dataset Documentatio